Introduction to simulation

- The Nature of Simulation
- Systems, Models, and Simulation

- Discrete-Event Simulation
- Advantages, Disadvantages, and Pitfalls of Simulation

- Exercise: design a simple simulator for RM scheduling
- Project: design a simulator for a Hierarchical Scheduling System

The Nature of Simulation

- *Simulation*: imitate the operations of *system*, usually using a computer
  - To study the system, often make assumptions/approximations, both logical and mathematical, about how it works
  - These assumptions form a *model* of the system
  - If model structure is simple enough, could use mathematical methods to get exact information on questions of interest — *analytical solution*

The Nature of Simulation, cont.

- But most complex systems require models that are also complex (to be valid)
  - Must be studied via simulation — evaluate model numerically and collect data to estimate model characteristics
- Example: Manufacturing company considering extending its plant
  - Build it and see if it works out?
  - Simulate current, expanded operations — could also investigate many other issues along the way, quickly and cheaply

The Nature of Simulation, cont.

- Some (not all) application areas
  - Systems engineering: supporting design decisions
  - Designing and analyzing manufacturing systems
  - Determining hardware requirements or protocols for communications networks
  - Determining hardware and software requirements for a computer system
  - Designing and operating transportation systems such as airports, freeways, ports, and subways
  - Evaluating designs for service organizations such as call centers, fast-food restaurants, hospitals, and post offices
  - Reengineering of business processes
  - Determining ordering policies for an inventory system
  - Analyzing financial or economic systems

The Nature of Simulation, cont.

- Impediments to acceptance, use of simulation
  - Models of large systems are usually very complex
    - But now we have better modeling software … more general, flexible, and still (relatively) easy to use
  - Can consume a lot of computer time
    - But now we have faster, bigger, cheaper hardware to allow for much better studies than just a few years ago … this trend will continue
    - However, simulation will also continue to push the envelope on computing power; we ask more and more of our simulation models
  - Impression that simulation is "just programming"
    - There's a lot more to a simulation study than just "coding" a model in some software and running it to get "the answer"
    - Need careful design and analysis of simulation models

Systems, Models and Simulation

- *System*: A collection of entities that act and interact together toward some end (Schmidt and Taylor, 1970)
  - In practice, depends on objectives of study
  - What are the boundaries of the system?
  - Judgment call: level of detail (e.g., what is an entity?)
  - Usually assume a time element – *dynamic* system

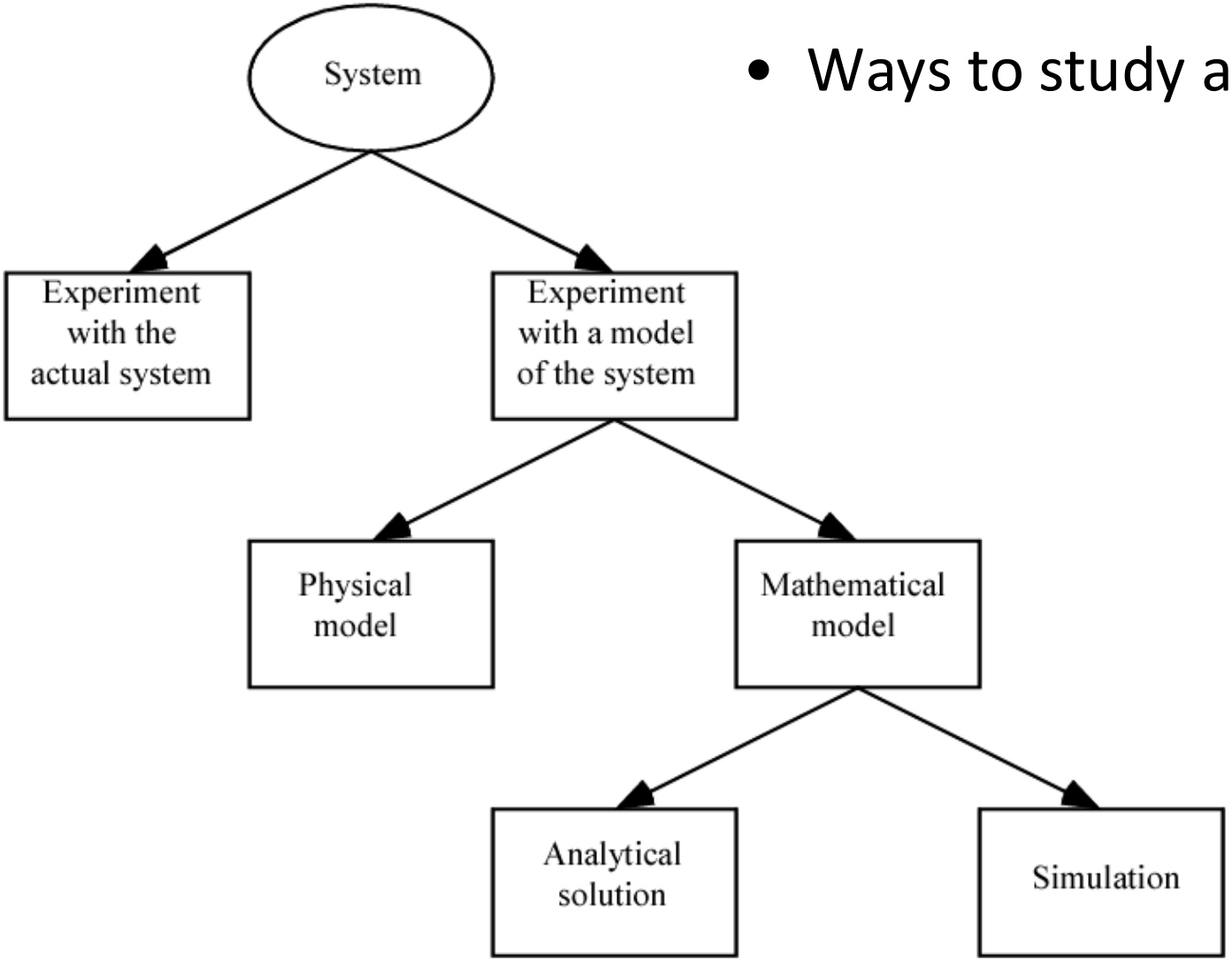
Systems, Models and Simulation, cont.

- *State* of a system: Collection of variables and their values necessary to describe the system at that time
  - Might depend on desired objectives, output performance measures
  - Bank model: Could include number of busy tellers, time of arrival of each customer, etc.

Systems, Models, and Simulation, cont.

- Types of systems
  - *Discrete*
    - State variables change instantaneously at separated points in time
    - Bank model: State changes occur only when a customer arrives or departs
  - *Continuous*
    - State variables change continuously as a function of time
    - Airplane flight: State variables like position, velocity change continuously
- Many systems are partly discrete, partly continuous

Systems, Models, and Simulation, cont.



- Ways to study a system

Systems, Models, and Simulation, cont.

- Classification of simulation models
  - *Static* vs. *dynamic*
  - *Deterministic* vs. *stochastic*
  - *Continuous* vs. *discrete*
- Most operational models are dynamic, stochastic, and discrete – will be called *discrete-event simulation models*

Discrete-even simulation

- *Discrete-event simulation*: Modeling of a system as it evolves over time by a representation where the state variables change instantaneously at separated points in time
  - State can change at only a *countable* number of points in time
  - These points in time are when *events* occur

- *Event*: Instantaneous occurrence that <u>may</u> change the state of the system

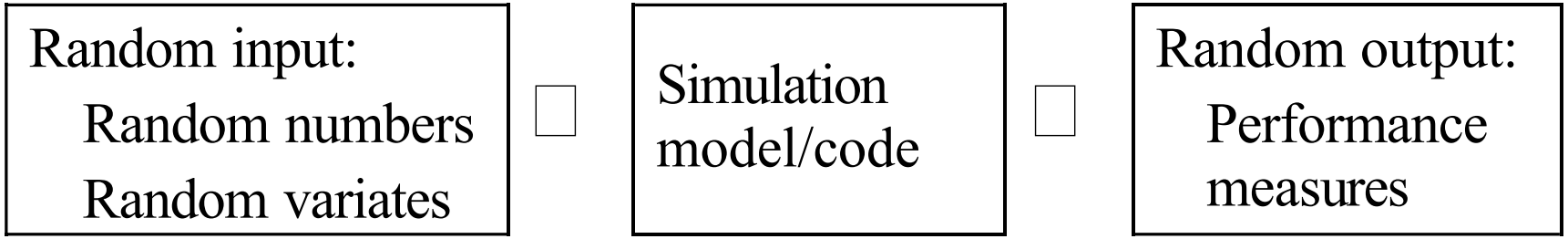
- Can in principle be done by hand; usually done on a computer

Simulation input

- Inappropriate input distribution(s) can lead to incorrect output, bad decisions

Use	**Pros**	**Cons**
Trace-driven Use actual data values to drive simulation	Valid *vis à vis* real world Direct	Not generalizable
Empirical distribution Use data values to define a "connect-the-dots" distribution (several specific ways)	Fairly valid Simple Fairly direct	May limit range of generated variates (depending on form)
Fitted "standard" distribution Use data to fit a classical distribution (exponential, uniform, Poisson, etc.)	Generalizable—fills in "holes" in data	May not be valid May be difficult

Simulation output



- Performance measures:
  - Interpreting simulation output: statistical analysis of output data
  - Observations from the probability distribution of output

- Outputs
  - *Standard reports* for the estimated performance measures
  - *Customized reports, Histograms, Time plots, Databases, Correlation plots*
  - *Export individual model output observations* to other software packages

Components and Organization of a Discrete-Event Simulation Model

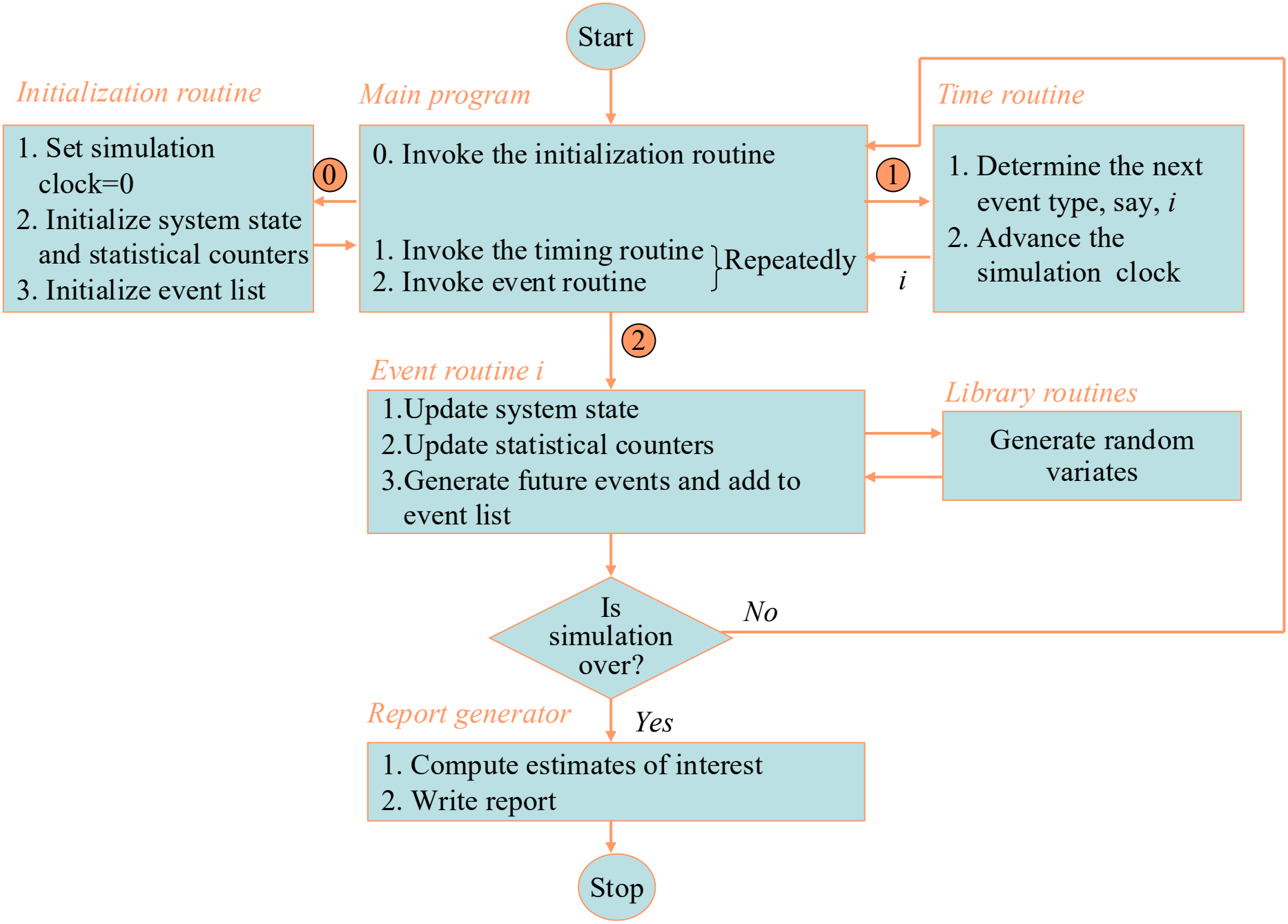
- Each simulation model must be customized to target system
- But there are several common components, general organization
  - *Systems state*: The collection of state variables necessary to describe the system at a particular time
  - *Simulation clock*: A variable giving the current value of simulated time
  - *Event list*: A list containing the next time when each type of event will occur
  - *Statistical counters*: Variables used for storing statistical information about system performance

Components and Organization of a Discrete-Event Simulation Model, cont.

- *Initialization routine*: A subprogram to initialize the simulation model at time 0
- *Timing routine*: A subprogram that determines the next event from the event list and then advances the simulation clock to the time when that event is to occur
- *Event routine*: A subprogram that updates the system state when a particular type of event occurs (there is one event routine for each event type)
- *Library routines*: A set of subprograms used to generate random observations from probability distributions that were determined as part of the simulation model

Components and Organization of a Discrete-Event Simulation Model, cont.

- *Report generator*: A subprogram that computes estimates (from the statistical counters) of the desired measures of performance and produces a report when the simulation ends
- *Main program*: A subprogram that invokes the timing routine to determine the next event and then transfers control to the corresponding event routine to update the system state appropriately. The main program may also check for termination and invoke the report generator when the simulation is over.


Start
Initialization routine
Main program
Time routine
1. Set simulation clock=0
2. Initialize system state and statistical counters
3. Initialize event list
0
0. Invoke the initialization routine
1. Invoke the timing routine
2. Invoke event routine
Repeatedly
1
i
1. Determine the next event type, say, i
2. Advance the simulation clock
2
Event routine i
Library routines
1.Update system state
2.Update statistical counters
3.Generate future events and add to event list
Generate random variates
Is simulation over?
No
Report generator
Yes
1. Compute estimates of interest
2. Write report
Stop

Time-Advance Mechanisms

- *Simulation clock*: Variable that keeps the current value of (simulated) time in the model
  - Must decide on, be consistent about, time units
  - Usually no relation between simulated time and (real) time needed to run a model on a computer

- Two approaches for time advance
  - *Next-event time advance* (usually used)
  - *Fixed-increment time advance* (seldom used)

Time-Advance Mechanisms, cont.

- More on next-event time advance
  - Initialize simulation clock to 0
  - Determine times of occurrence of future events – *event list*
  - Clock advances to next (most imminent) event, which is executed
    - Event execution may involve updating event list
  - Continue until stopping rule is satisfied
  - Clock "jumps" from one event time to the next, and doesn't "exist" for times between successive events … periods of inactivity are ignored

Advantages, Disadvantages, Pitfalls

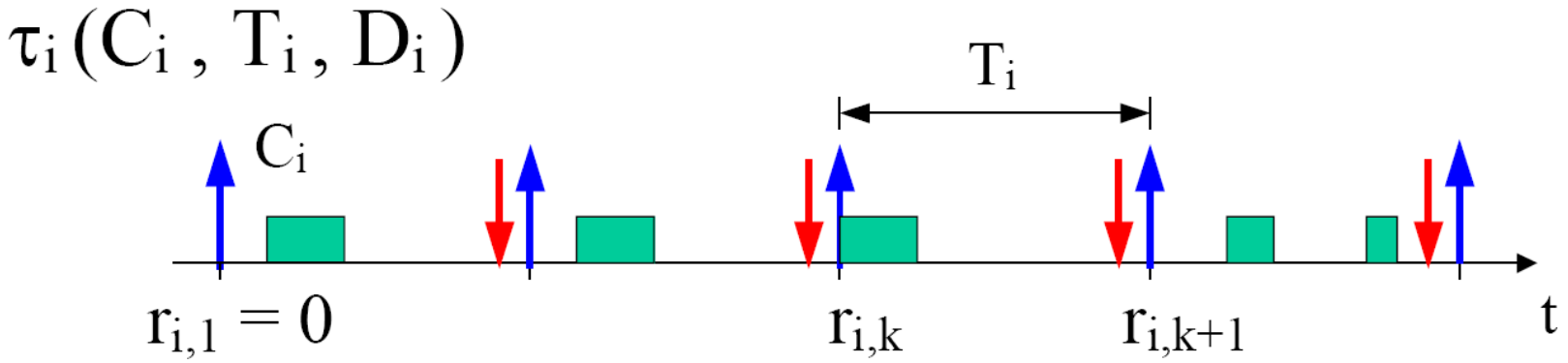
- Advantages
  - Simulation allows great flexibility in modeling complex systems, so simulation models can be highly valid
  - Easy to compare alternatives
  - Control experimental conditions
  - Can study system with a very long time frame
- Disadvantages
  - Stochastic simulations produce only estimates – with noise
  - Simulation models can be expensive to develop
  - Simulations usually produce large volumes of output – need to summarize, statistically analyze appropriately
- Pitfalls
  - Failure to identify objectives clearly up front
  - Inappropriate level of detail (both ways)
  - Inadequate design and analysis of simulation experiments
  - Inadequate education, training

Example simulation: RM

- Simulate by hand the following tasks running on a single processor using Rate Monotonic scheduling

Task	WCET	Period = Deadline
A	10	25
B	10	40
C	20	100

Exercise setup: RM scheduling



$D_i = T_i$

- **Rate Monotonic (RM):**

  $p_i \propto 1/T_i$ (static)

- **Earliest Deadline First (EDF):**

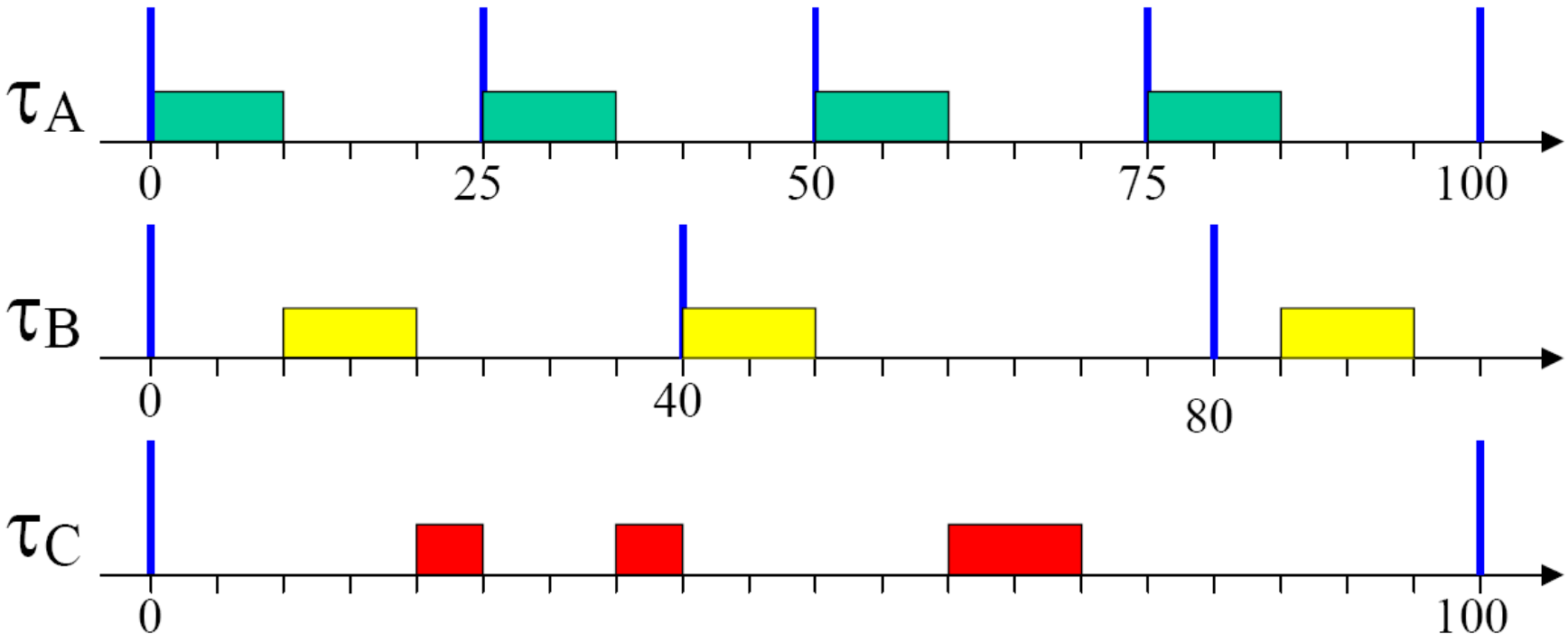
  $p_i \propto 1/d_i$ (dynamic) $\quad d_{i,k} = r_{i,k} + D_i$

Exercise setup: Assumptions

- Single processor
- Strictly periodic tasks
- Deadlines are equal to the periods
- All tasks are independent
- All tasks are released as soon as they arrive
- All tasks arrive at time zero

- Execution time is always equal to the WCET

Exercise: solution



RM simulator

- Design a program to simulate what you have simulated by hand

- What is the system state?
  - What are the entities?
  - What are their attributes?
- Event routine(s)
  - What are the events?
  - How do you organize your event list?
  - How do you update the system state when an event occurs?
- Timing routine:
  - How do you determine the next event?
  - How do you advance time?
- Report generator:
  - What information do you collect?
  - How do you report it?

Simple Solution

```
1  n = number of cycles {use a multiple of the LCM of periods}
2  initialize_priorities;
   {initialize_jobs}
3  for all τ_i ∈ Γ do
4     for all job_j; j = 1 to n / τ_i.period do
5        τ_i.job_j.release = τ_i.period × (j − 1)
6        τ_i.job_j.time = random(bcet, wcet)
7        insert_job(τ_i.job_j, jobs_queue)
8     end for
9  end for
10 sort(jobs_queue) on start time, priority
   {simulate}
11 for cycle = 1 to n do
12    ready_list = get_ready(job_list, cycle)
13    current_job = highest_priority(ready_list)
14    current_job.time−−;
15    if current_job.time == 0 then
16       current_job.response = cycle-start
17       remember worst-case response
18       remove(current_job, ready_list)
19    end if
20 end for
```